

Spring JDBC

Spring JDBC is spring module which is used for connect your java application with your database application.

Spring **Jdbc-Template** is a powerful mechanism to connect to the database and execute SQL queries. It internally uses JDBC api, but eliminates a lot of problems of JDBC API.

Problems of JDBC API

The problems of JDBC API are as follows:

- We need to write a lot of code before and after executing the query, such as creating connection, statement, closing resultset, connection etc.
- We need to perform exception handling code on the database logic.
- We need to handle transaction.
- Repetition of all these codes from one to another database logic is a time consuming task.

Advantage of Spring Jdbc-Template

Spring Jdbc-Template eliminates all the above mentioned problems of JDBC API. It provides you methods to write the queries directly, so it saves a lot of work and time.

Q. Why we need to use the spring JDBC if we have plain JDBC?

1. Not need to handle check exceptions.
2. Not need to write boilerplate code like as connection establish, driver loading, statement creation etc
It is internally handle by Spring JDBC.

3. Not need to close the connection manually etc.

Q. what is diff between plain JDBC and spring JDBC?

Standard JDBC or Plain JDBC	Spring JDBC
In Plain JDBC we must be handle checked exception	Spring JDBC not need to handle checked exception
In JDBC we have to write boiler plate code manually	Spring JDBC provide Centralize facility to write connection code in single time by using xml or annotation programmer not need to write connection code every time
In Standard JDBC we need to close connection manually and clean resources	In Spring JDBC we not need to close connection manually and clean resource it is responsibility of spring framework
Standard JDBC not provide vendor specific error message to us	Spring JDBC provide vendor specific message to us
Standard JDBC or Plain JDBC is root api of database connection in java	Spring JDBC is built on top up of Standard JDBC means if user known about standard jdbc then he can easily work with spring jdbc
Standard Jdbc Provide PreparedStatement for perform DDL and DML	Spring JDBC provide JdbcTemplate for perform DDL and DML and internally use PreparedStatement interface

Steps to work with spring JDBC

1. Create Maven project

2. Add following maven dependencies

- i) Spring core
- ii) Spring jdbc
- iii) Spring context
- iv) Mysql connector

```
<dependencies>
  <!-- https://mvnrepository.com/artifact/org.springframework/spring-core -->
  <dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-core</artifactId>
    <version>5.3.17</version>
  </dependency>
  <!-- https://mvnrepository.com/artifact/org.springframework/spring-context -->
  <dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-context</artifactId>
    <version>5.3.17</version>
  </dependency>
  <!-- https://mvnrepository.com/artifact/org.springframework/spring-jdbc -->
  <dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-jdbc</artifactId>
    <version>5.3.17</version>
  </dependency>
  <!-- https://mvnrepository.com/artifact/com.mysql/mysql-connector-j -->
  <dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <version>8.0.33</version>
  </dependency>
</dependencies>
```

3. Configure database using XML or annotations

if we want to configure database credential using XML we need to write following bean

```
<bean id="ref" class="org.springframework.jdbc.datasource.DriverManagerDataSource">
  <property name="driverClassName" value="com.mysql.cj.jdbc.Driver"/>
</bean>
```

```

    <property name="username" value="root"/>
    <property name="password" value="root"/>
    <property name="url" value="jdbc:mysql://localhost:3306/mysql"/>
</bean>

```

Example with source code

test.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE beans PUBLIC "-//SPRING//DTD BEAN 2.0//EN"
    "https://www.springframework.org/dtd/spring-beans-2.0.dtd">

<beans>
    <bean id="conn" class="org.springframework.jdbc.datasource.DriverManagerDataSource">
        <property name="driverClassName" value="com.mysql.cj.jdbc.Driver"/>
        <property name="username" value="root"/>
        <property name="password" value="root"/>
        <property name="url" value="jdbc:mysql://localhost:3306/mysql"/>
    </bean>
</beans>

```

LoadingApplication.java

```

package org.techhub;

import org.springframework.beans.factory.BeanFactory;
import org.springframework.beans.factory.xml.XmlBeanFactory;
import org.springframework.core.io.ClassPathResource;
import org.springframework.core.io.Resource;
import org.springframework.jdbc.datasource.DriverManagerDataSource;
public class LoadingDBApplication {
    public static void main(String[] args) {
        Resource r=new ClassPathResource("test.xml");
        BeanFactory bf= new XmlBeanFactory(r);
        DriverManagerDataSource d=(DriverManagerDataSource)bf.getBean("conn");
        if(d!=null) {
            System.out.println("Database is connected");
        }
        else {
            System.out.println("Database is not connected");
        }
    }
}

```

4. Create JdbcTemplate reference: JdbcTemplate is class from org.springframework.jdbc.core package and it is used for perform all DDL and DML operation on database like as PreparedStatement interface. JdbcTemplate is dependent on DriverManagerDataSource means we can say we need to call bean of DriverManagerDataSource through the JdbcTemplate

Internal code

```
class JdbcTemplate
{
    void setDataSource(Driver driver)
    {
    }
}
```

Example to configure JdbcTemplate by using XML

test.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE beans PUBLIC "-//SPRING//DTD BEAN 2.0//EN"
    "https://www.springframework.org/dtd/spring-beans-2.0.dtd">

<beans>
<bean id="conn" class="org.springframework.jdbc.datasource.DriverManagerDataSource">
    <property name="driverClassName" value="com.mysql.cj.jdbc.Driver"/>
    <property name="username" value="root"/>
    <property name="password" value="root"/>
    <property name="url" value="jdbc:mysql://localhost:3306/mysql"/>
</bean>
<bean id="template" class="org.springframework.jdbc.core.JdbcTemplate">
    <property name="dataSource" ref="conn"/>
</bean>
</beans>
```

LoadingDBApplication.java

```
package org.techhub;
import org.springframework.beans.factory.BeanFactory;
import org.springframework.beans.factory.xml.XmlBeanFactory;
import org.springframework.core.io.ClassPathResource;
import org.springframework.core.io.Resource;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.jdbc.datasource.DriverManagerDataSource;
public class LoadingDBApplication {
    public static void main(String[] args) {
        Resource r=new ClassPathResource("test.xml");
```

```

        BeanFactory bf= new XmlBeanFactory(r);
        JdbcTemplate template=(JdbcTemplate)bf.getBean("template");
        System.out.println("Database is connected and template get executed");
    }
}

```

5. Work with database: JdbcTemplate provide the some inbuilt method to us to work with database like as inserting record in database, deleting record from database etc.

void execute(String sqlStatement): this method is used for execute the all DDL and DML operation on database except select.

Example:

```

package org.techhub;
import org.springframework.beans.factory.BeanFactory;
import org.springframework.beans.factory.xml.XmlBeanFactory;
import org.springframework.core.io.ClassPathResource;
import org.springframework.core.io.Resource;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.jdbc.datasource.DriverManagerDataSource;
public class LoadingDBApplication {
    public static void main(String[] args) {
        Resource r = new ClassPathResource("test.xml");
        BeanFactory bf = new XmlBeanFactory(r);
        JdbcTemplate template = (JdbcTemplate) bf.getBean("template");
        System.out.println("Database is connected and template get executed");
        template.execute("insert into register values('abc','abc@gmail.com','123456')");
        System.out.println("Success.....");
    }
}

```

If we use the execute() method and if we want to pass run time parameter to SQL Statement we need to handle the SQL Injection.

```

package org.techhub;
import org.springframework.beans.factory.BeanFactory;
import org.springframework.beans.factory.xml.XmlBeanFactory;
import org.springframework.core.io.ClassPathResource;
import org.springframework.core.io.Resource;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.jdbc.datasource.DriverManagerDataSource;
import java.util.*;

```

```

public class LoadingDBApplication {
    public static void main(String[] args) {
        Scanner xyz = new Scanner(System.in);
        Resource r = new ClassPathResource("test.xml");
        BeanFactory bf = new XmlBeanFactory(r);
        JdbcTemplate template = (JdbcTemplate) bf.getBean("template");
        System.out.println("Database is connected and template get executed");
        System.out.println("Enter name email and contact");
        String name=xyz.nextLine();
        String email=xyz.nextLine();
        String contact=xyz.nextLine();
        template.execute("insert into register values('"+name+"','"+email+"','"+contact+"')");
        System.out.println("Success.....");
    }
}

```

If we want to avoid SQL Injection for pass run time parameter to SQL Statement we have update() method of JdbcTemplate.

int update(String sqlStatement, PreparedStatementSetter): this method can perform all DDL and DML operation on database using PreparedStatementSetter interface this method return 1 if operation executed successfully otherwise return 0

Example with source code

```

package org.techhub;

import org.springframework.beans.factory.BeanFactory;
import org.springframework.beans.factory.xml.XmlBeanFactory;
import org.springframework.core.io.ClassPathResource;
import org.springframework.core.io.Resource;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.jdbc.core.PreparedStatementSetter;
import org.springframework.jdbc.datasource.DriverManagerDataSource;

import java.sql.PreparedStatement;
import java.sql.SQLException;
import java.util.*;

public class LoadingDBApplication {
    public static void main(String[] args) {
        Scanner xyz = new Scanner(System.in);
        Resource r = new ClassPathResource("test.xml");
        BeanFactory bf = new XmlBeanFactory(r);
        JdbcTemplate template = (JdbcTemplate) bf.getBean("template");
        System.out.println("Database is connected and template get executed");
        System.out.println("Enter name email and contact");
    }
}

```

```

        final String name=xyz.nextLine();
        final String email=xyz.nextLine();
        final String contact=xyz.nextLine();
        PreparedStatementSetter pstmt=new PreparedStatementSetter() {

            public void setValues(PreparedStatement ps) throws SQLException {
                ps.setString(1,name);
                ps.setString(2,email);
                ps.setString(3, contact);
            }
        };
        int value=template.update("insert into register values(?,?,?)", pstmt);
        System.out.println("Success.....");
    }
}
or
package org.techhub;

import org.springframework.beans.factory.BeanFactory;
import org.springframework.beans.factory.xml.XmlBeanFactory;
import org.springframework.core.io.ClassPathResource;
import org.springframework.core.io.Resource;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.jdbc.core.PreparedStatementSetter;
import org.springframework.jdbc.datasource.DriverManagerDataSource;

import java.sql.PreparedStatement;
import java.sql.SQLException;
import java.util.*;
public class LoadingDBApplication {
    public static void main(String[] args) {
        Scanner xyz = new Scanner(System.in);
        Resource r = new ClassPathResource("test.xml");
        BeanFactory bf = new XmlBeanFactory(r);
        JdbcTemplate template = (JdbcTemplate) bf.getBean("template");
        System.out.println("Database is connected and template get executed");
        System.out.println("Enter name email and contact");
        final String name=xyz.nextLine();
        final String email=xyz.nextLine();
        final String contact=xyz.nextLine();
        int value=template.update("insert into register values(?,?,?)",new PreparedStatementSetter() {

```

```

        public void setValues(PreparedStatement ps) throws SQLException {
            ps.setString(1,name);
            ps.setString(2,email);
            ps.setString(3, contact);
        }
    });
    System.out.println("Success.....");
}
}

```

Example: delete record from database table using is email

```

package org.techhub;
import org.springframework.beans.factory.BeanFactory;
import org.springframework.beans.factory.xml.XmlBeanFactory;
import org.springframework.core.io.ClassPathResource;
import org.springframework.core.io.Resource;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.jdbc.core.PreparedStatementSetter;
import org.springframework.jdbc.datasource.DriverManagerDataSource;
import java.sql.PreparedStatement;
import java.sql.SQLException;
import java.util.*;
public class LoadingDBApplication {
    public static void main(String[] args) {
        Scanner xyz = new Scanner(System.in);
        Resource r = new ClassPathResource("test.xml");
        BeanFactory bf = new XmlBeanFactory(r);
        JdbcTemplate template = (JdbcTemplate) bf.getBean("template");
        System.out.println("Database is connected and template get executed");
        System.out.println("Enter email ");
        final String email=xyz.nextLine();
        int value=template.update("delete from register where email=?",new PreparedStatementSetter() {
            public void setValues(PreparedStatement ps) throws SQLException {
                ps.setString(1,email);
            }
        });
        System.out.println("Success.....");
    }
}

```

Example for update the record using Spring JDBC


```

package org.techhub;
import org.springframework.beans.factory.BeanFactory;
import org.springframework.beans.factory.xml.XmlBeanFactory;
import org.springframework.core.io.ClassPathResource;
import org.springframework.core.io.Resource;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.jdbc.core.PreparedStatementSetter;
import org.springframework.jdbc.datasource.DriverManagerDataSource;
import java.sql.PreparedStatement;
import java.sql.SQLException;
import java.util.*;

public class LoadingDBApplication {
    public static void main(String[] args) {
        Scanner xyz = new Scanner(System.in);
        Resource r = new ClassPathResource("test.xml");
        BeanFactory bf = new XmlBeanFactory(r);
        JdbcTemplate template = (JdbcTemplate) bf.getBean("template");
        System.out.println("Database is connected and template get executed");
        System.out.println("Enter name email contact");
        final String name=xyz.nextLine();
        final String email=xyz.nextLine();
        final String contact=xyz.nextLine();

        int value=template.update("update register set name=?,contact=? where email=?",new
PreparedStatementSetter() {

                public void setValues(PreparedStatement ps) throws SQLException {
                    ps.setString(1, name);
                    ps.setString(2, contact);
                    ps.setString(3, email);
                }
            });
        System.out.println("Success.....");
    }
}

```

How to configure Spring JDBC by using annotations.

If we want to configure Spring JDBC by using annotation we have to create Configuration class and configure the bean for DriverManagerDataSource and JdbcTemplate and call JdbcTemplate bean where we want to perform database operation.

Config.java

```
package org.techhub;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.jdbc.datasource.DriverManagerDataSource;

@Configuration
public class Config {
    @Bean(name="dataSource")
    public DriverManagerDataSource getDataSource() {
        DriverManagerDataSource dataSource=new DriverManagerDataSource();
        dataSource.setDriverClassName("com.mysql.cj.jdbc.Driver");
        dataSource.setUsername("root");
        dataSource.setPassword("root");
        dataSource.setUrl("jdbc:mysql://localhost:3306/mysql");
        return dataSource;
    }
    @Bean(name="template")
    public JdbcTemplate getTemplate() {
        JdbcTemplate template=new JdbcTemplate();
        template.setDataSource(getDataSource());
        return template;
    }
}
```

ClientApplication.java

```
package org.techhub;

import org.springframework.context.annotation.AnnotationConfigApplicationContext;
import org.springframework.jdbc.core.JdbcTemplate;
public class AnnotationClientApplication {
    public static void main(String[] args) {
        AnnotationConfigApplicationContext context= new
AnnotationConfigApplicationContext(Config.class);
        JdbcTemplate template=(JdbcTemplate)context.getBean("template");
        template.execute("insert into register values('pqr','pqr@gmail.com','1122334455')");
        System.out.println("Success.....");
    }
}
```

How to fetch or select data from database table using Spring JDBC

If we want to fetch data from database table using Spring JDBC we have to use following steps.

1. Create Maven Project and following dependencies

```
<dependencies>
    <!-- https://mvnrepository.com/artifact/org.springframework/spring-core -->
    <dependency>
        <groupId>org.springframework</groupId>
        <artifactId>spring-core</artifactId>
        <version>5.3.17</version>
    </dependency>
    <!-- https://mvnrepository.com/artifact/org.springframework/spring-context -->
    <dependency>
        <groupId>org.springframework</groupId>
        <artifactId>spring-context</artifactId>
        <version>5.3.17</version>
    </dependency>
    <!-- https://mvnrepository.com/artifact/org.springframework/spring-jdbc -->
    <dependency>
        <groupId>org.springframework</groupId>
        <artifactId>spring-jdbc</artifactId>
        <version>5.3.17</version>
    </dependency>
    <!-- https://mvnrepository.com/artifact/com.mysql/mysql-connector-j -->
    <dependency>
        <groupId>com.mysql</groupId>
        <artifactId>mysql-connector-j</artifactId>
        <version>8.0.33</version>
    </dependency>
</dependencies>
```

2. Configure DriverManagerDataSource and JdbcTemplate using XML or Annotation

```
package org.techhub;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.jdbc.datasource.DriverManagerDataSource;
@Configuration
public class DBConfig {
    @Bean(name="dataSource")
    public DriverManagerDataSource getDataSource() {
        DriverManagerDataSource dataSource = new DriverManagerDataSource();
```

```

        dataSource.setDriverClassName("com.mysql.cj.jdbc.Driver");
        dataSource.setUsername("root");
        dataSource.setPassword("root");
        dataSource.setUrl("jdbc:mysql://localhost:3306/mysql");
        return dataSource;
    }
    @Bean(name="template")
    public JdbcTemplate getTemplate() {
        JdbcTemplate template=new JdbcTemplate();
        template.setDataSource(this.getDataSource());
        return template;
    }
}

```

3. Create Client Application and call template by using getBean()

```

package org.techhub;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;
import org.springframework.jdbc.core.JdbcTemplate;
public class ClientApplication {
    public static void main(String[] args) {
        AnnotationConfigApplicationContext context=new
AnnotationConfigApplicationContext(DBConfig.class);
        JdbcTemplate template=(JdbcTemplate)context.getBean("template");

    }
}

```

4. call query method of JdbcTemplate to fetch data from database table

Note: query() method work as exeucteQuery() method of PreparedStatement.

Syntax:

List query(String selectStatement, RowMapper):

String selectStatement: here we can write database select query as parameter

RowMapper: RowMapper is interface from org.springframework.jdbc.core package and it is used for fetch data from database table and this interface provide mapRow() method to us which help us to fetch data from database and mapRow() contain ResultSet interface as parameter which is used for fetch data from database table

Method of RowMapper interface

It defines only one method mapRow that accepts ResultSet instance and int as the parameter list.

The `int rowNum` parameter in the `mapRow` method is used to indicate the current row number of the `ResultSet` being processed. This parameter is particularly useful when you need to keep track of the row index for operations that might depend on the specific row number, such as logging, handling specific rows differently, or maintaining order in collections that are populated during the mapping process.

Example with source code

In this example create **Register (POJO) class**

```
package org.techhub;

import org.springframework.context.annotation.AnnotationConfigApplicationContext;
import org.springframework.jdbc.core.ColumnMapRowMapper;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.jdbc.core.RowMapper;
import org.springframework.jdbc.core.RowMapperResultSetExtractor;

import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.*;

public class ClientApplication {
    public static void main(String[] args) {
        AnnotationConfigApplicationContext context =
            new AnnotationConfigApplicationContext(DBConfig.class);
        JdbcTemplate template = (JdbcTemplate) context.getBean("template");
        RowMapper<Register> rowMapper = new RowMapper<Register>() {
            public Register mapRow(ResultSet rs, int rowNum) throws SQLException {

                Register r = new Register();
                r.setName(rs.getString(1));
                r.setEmail(rs.getString(2));
                r.setContact(rs.getString(3));
                return r;
            }
        };
        List<Register> list = template.query("select *from register", rowMapper);
        for (Register r : list) {
            System.out.println(r.getName()+"\t"+r.getEmail()+"\t"+r.getContact());
        }
    }
}
```

How to pass parameter to select query using Spring JDBC

If we want to pass parameter to select query we have to use following version of query() method of JdbcTemplate.

Syntax:

List query(String selectStatement, PreparedStatementSetter, RowMapper):

String selectStatement: this parameter is used for write select query

PreparedStatementSetter: this parameter is used for pass parameter to select query after where clause

RowMapper: this parameter helps us to fetch data from database table.

Example: we want to input email from keyboard and fetch user data.

```
package org.techhub;

import org.springframework.context.annotation.AnnotationConfigApplicationContext;
import org.springframework.jdbc.core.ColumnMapRowMapper;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.jdbc.core.PreparedStatementSetter;
import org.springframework.jdbc.core.RowMapper;
import org.springframework.jdbc.core.RowMapperResultSetExtractor;

import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.*;

public class ClientApplication {
    public static void main(String[] args) {
        Scanner xyz = new Scanner(System.in);
        System.out.println("Enter email of user");
        final String email = xyz.nextLine();
        AnnotationConfigApplicationContext context =
            new AnnotationConfigApplicationContext(DBConfig.class);
        JdbcTemplate template = (JdbcTemplate) context.getBean("template");
        RowMapper<Register> rowMapper = new RowMapper<Register>() {
            public Register mapRow(ResultSet rs, int rowNum) throws SQLException {
                // TODO Auto-generated method stub
                Register r = new Register();
                r.setName(rs.getString(1));
                r.setEmail(rs.getString(2));
                r.setContact(rs.getString(3));
            }
        };
    }
}
```

```

        return r;
    }
};
PreparedStatementSetter stmt=new PreparedStatementSetter() {
    public void setValues(PreparedStatement ps) throws SQLException {
        ps.setString(1, email);
    }
};
List<Register> list = template.query("select *from register where email=?", stmt, rowMapper);
for (Register r : list) {
    System.out.println(r.getName()+"\t"+r.getEmail()+"\t"+r.getContact());
}
}
}

```

Above code Using JDK1.8 version and Lamda expression

```

package org.techhub;
import org.springframework.context.ConfigurableApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.jdbc.core.PreparedStatementSetter;
import org.springframework.jdbc.core.RowMapper;
import org.springframework.jdbc.datasource.DriverManagerDataSource;

import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.*;

public class DbConnectApp {
    public static void main(String[] args) {
        ConfigurableApplicationContext context=new ClassPathXmlApplicationContext("conn.xml");
        JdbcTemplate template=(JdbcTemplate)context.getBean("template");
        List<Register> list=template.query("select *from sepspringjdbc", (ResultSet rs, int rowNum)->new
Register(rs.getString("name"),rs.getString("email"),rs.getString("contact")));
        list.forEach((r)->System.out.println(r.getName()+"\t"+r.getEmail()+"\t"+r.getContact()));
    }
}

```

CrudApplication by using Spring JDBC

Now we want to create simple application for store book information in database table and perform different operation on it.

Suppose consider we have one table name as Book

Bookid	Name	Author	Publication	price
--------	------	--------	-------------	-------

1. Add New Book
2. View All Books
3. Search Book by Name or Author or publication
4. Delete Book by Name
5. Update book info by id

Flow of application

Client Application ----- model ----- service ----- repository ----- database.

Table

```
mysql> create table book(bookid int(5) primary key auto_increment, name varchar(200),author  
varchar(200),pub varchar(200),price int(5));
```

Query OK, 0 rows affected, 2 warnings (0.07 sec)

```
mysql> commit;
```

Query OK, 0 rows affected (0.01 sec)

Steps to Create Application

1) Create Maven project

2) Add Following Maven Dependency

i) spring ii) spring context iii) spring JDBC iv) mysql connector

3) Create Model class

```
package org.techhub.model;
```

```
public class Book {
```

```
    private int id;
```

```
    private String name;
```

```
    private String author;
```

```
    private String pub;
```

```
    private int price;
```

```
    public Book() {
```

```
    }
```

```
    public Book(String name,String author,String pub,int price) {
```

```
        this.name=name;
```

```
        this.author=author;
```



```
        this.pub=pub;
        this.price=price;
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getAuthor() {
        return author;
    }

    public void setAuthor(String author) {
        this.author = author;
    }

    public String getPub() {
        return pub;
    }

    public void setPub(String pub) {
        this.pub = pub;
    }

    public int getPrice() {
        return price;
    }

    public void setPrice(int price) {
        this.price = price;
    }
}
```

```
}
```

4) Create Service layer: if we want to create service class we have @Service annotations.

import org.springframework.stereotype.Service;

@Service("bookService")

public class BookService {

```
}
```

5) Create Repository layer: repository class means a class which contain database logics called as repository class.

package org.techhub.repo;

import org.springframework.stereotype.Repository;

@Repository("bookRepo")

public class BookRepo {

```
}
```

6) Create Configuration class and configure bean for DriverManagerDataSource as well as Configure bean for JdbcTemplate and use @ComponentScan annotation and mention all service and repository packages name in that so spring can identity service layers and repository layer of application

package org.techhub.config;

import org.springframework.context.annotation.Bean;

import org.springframework.context.annotation.ComponentScan;

import org.springframework.context.annotation.Configuration;

import org.springframework.jdbc.core.JdbcTemplate;

import org.springframework.jdbc.datasource.DriverManagerDataSource;

@Configuration

@ComponentScan(basePackages = {"org.techhub.model, org.techhub.repo, org.techhub.service"})

public class ConfigApp {

@Bean(name="dataSource")

public DriverManagerDataSource getDataSource() {

DriverManagerDataSource datasource= **new** DriverManagerDataSource();

datasource.setUsername("root");

datasource.setPassword("root");

datasource.setDriverClassName("com.mysql.cj.jdbc.Driver");

datasource.setUrl("jdbc:mysql://localhost:3306/mysql");

return datasource;

```
}
```

@Bean(name="template")

```

        public JdbcTemplate getTemplate() {
            JdbcTemplate template= new JdbcTemplate();
            template.setDataSource(getDataSource());
            return template;
        }
    }
}

```

Create Object of JdbcTemplate in Repository class using @Autowired annotation

```

@Repository("bookRepo")
public class BookRepo {
    @Autowired
    JdbcTemplate template;
}

```

Create object of Repository in service class using @AutoWired Annotation

```

@Service("bookService")
public class BookService {

    @Autowired
    BookRepo bookRepo;

}

```

Call Service class in ClientApplication

```

ClientApplication.java
public class ClientApplication {
    public static void main(String[] args) {
        AnnotationConfigApplicationContext context= new
        AnnotationConfigApplicationContext(ConfigApp.class);
        BookService bookService=(BookService)context.getBean("bookService");
    }
}

Service layer
@Service("bookService")
public class BookService {
    @Autowired
    BookRepo bookRepo;
}

@Repository("bookRepo")
public class BookRepo {
    @Autowired
    JdbcTemplate template;
}

@Configuration
@ComponentScan(basePackages = {"org.techhub.model,org.techhub.repo,org.techhub.service"})
public class ConfigApp {
    @Bean(name="dataSource")
    public DriverManagerDataSource getDataSource() {
        DriverManagerDataSource datasource= new DriverManagerDataSource();
        datasource.setUsername("root");
        datasource.setPassword("root");
        datasource.setDriverClassName("com.mysql.cj.jdbc.Driver");
        datasource.setUrl("jdbc:mysql://localhost:3306/mysql");
        return datasource;
    }
}

@Bean(name="template")
public JdbcTemplate getTemplate() {
    JdbcTemplate template= new JdbcTemplate();
    template.setDataSource(getDataSource());
    return template;
}

Create object of
repository class
Create object of
template class
Create object of
data source class.

```

The diagram illustrates the object creation flow in a Spring application. It shows four main components: ClientApplication, BookService (Service layer), BookRepo (Repository), and ConfigApp (Configuration). Arrows indicate the sequence of object creation: ClientApplication creates BookService, which creates BookRepo, which creates JdbcTemplate, which creates DriverManagerDataSource. The annotations @Service, @Repository, @Autowired, and @Configuration are used to configure the objects.

above diagram indicate we configure complete project and its layer just we want to perform database operation on it

case 1: add new book
case 2: view all book
case 3: Search book
case 4: delete book
case 5: update book

if we want to add new book then create one method in Repository and pass model object in it which is Book class.

```
package org.techhub.repo;
import java.sql.PreparedStatement;
import java.sql.SQLException;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.jdbc.core.PreparedStatementSetter;
import org.springframework.stereotype.Repository;
import org.techhub.model.Book;
@Repository("bookRepo") //bookRepo work as reference or bean id
public class BookRepo {
    @Autowired
    JdbcTemplate template;

    public boolean isAddBook(final Book book) {
        //write here insert logic using template.
        /*PreparedStatementSetter pstmt= new PreparedStatementSetter() {

            public void setValues(PreparedStatement ps) throws SQLException {

                ps.setString(1,book.getName());
                ps.setString(2, book.getAuthor());
                ps.setString(3, book.getPub());
                ps.setInt(4,book.getPrice());
            }
        };*/
        //below code is lamda version of above code.
        PreparedStatementSetter pstmt= (PreparedStatement ps)->{

            ps.setString(1,book.getName());
            ps.setString(2, book.getAuthor());
            ps.setString(3, book.getPub());
            ps.setInt(4,book.getPrice());
        };
    }
}
```

```

        int value=template.update("insert into book values('0',?,?,?,?)", pstmt);
        return value>0?true:false;
    }
}

```

once create repository class we have to call method of Repository in Service and pass model class.

```

package org.techhub.service;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.techhub.model.Book;
import org.techhub.repo.BookRepo;
@Service("bookService")
public class BookService {
    @Autowired
    BookRepo bookRepo;
    public boolean isAddBook(Book book) {
        //boolean b= bookRepo.isAddBook(book);
        //return b;
        return bookRepo.isAddBook(book);
    }
}

```

once provide model object from a service to repository after that we have to create object of model in client application and store data in it and pass model object from client application to service layer so finally our flow maintain like client application ----- model ---- service --- repository ---template --- datasource and then finally in database.

```

package org.techhub.client;

import org.springframework.context.annotation.AnnotationConfigApplicationContext;
import org.techhub.config.ConfigApp;
import org.techhub.model.Book;
import org.techhub.service.BookService;
import java.util.*;
public class ClientApplication {
    public static void main(String[] args) {

        AnnotationConfigApplicationContext context= new
            AnnotationConfigApplicationContext(ConfigApp.class);
        BookService bookService=(BookService)context.getBean("bookService");
        do {
            Scanner xyz = new Scanner(System.in);
            System.out.println("1:Add New Book");

```

```

        System.out.println("2:View All Book");
        System.out.println("Enter your choice");
        int choice=xyz.nextInt();
        switch(choice) {
        case 1:
            xyz.nextLine();
            System.out.println("Enter name author publication and price of book");
            String name=xyz.nextLine();
            String author=xyz.nextLine();
            String pub=xyz.nextLine();
            int price=xyz.nextInt();
            Book b = new Book(name,author,pub,price);
            boolean b1= bookService.isAddBook(b);
            if(b1) {
                System.out.println("New Book Added");
            }
            else {
                System.out.println("Some problem is there.....");
            }
            break;
        case 2:
            List<Book>
list=bookService.getAllBook();list.forEach((book)->Syste
m.out.println(book.getName()+"\t"+book.getAuthor()+"\t"+book.getPrice()));
            break;
        case 3:
            break;
        case 4:
            break;
        case 5:
            break;
        case 6:
            System.exit(0);
            break;
        default:
            System.out.println("Wrong choice");
        }
    }while(true);
}
}

```

Complete Source code

Configuration class

```
package org.techhub.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.jdbc.datasource.DriverManagerDataSource;

@Configuration
@ComponentScan(basePackages = {"org.techhub.model,org.techhub.repo,org.techhub.service"})
public class ConfigApp {
    @Bean(name="dataSource")
    public DriverManagerDataSource getDataSource() {
        System.out.println("Data Source call from template");
        DriverManagerDataSource datasource= new DriverManagerDataSource();
        datasource.setUsername("root");
        datasource.setPassword("root");
        datasource.setDriverClassName("com.mysql.cj.jdbc.Driver");
        datasource.setUrl("jdbc:mysql://localhost:3306/mysql");
        return datasource;
    }
    @Bean(name="template")
    public JdbcTemplate getTemplate() {

        JdbcTemplate template= new JdbcTemplate();
        template.setDataSource(getDataSource());
        System.out.println("Template call from repository");
        return template;
    }
}
```

Model class

```
package org.techhub.model;

public class Book {
    private int id;
    private String name;
    private String author;
    private String pub;
    private int price;
```

```
public Book() {  
  
}  
public Book(String name,String author,String pub,int price) {  
    this.name=name;  
    this.author=author;  
    this.pub=pub;  
    this.price=price;  
}  
  
    public int getId() {  
        return id;  
    }  
  
    public void setId(int id) {  
        this.id = id;  
    }  
  
    public String getName() {  
        return name;  
    }  
  
    public void setName(String name) {  
        this.name = name;  
    }  
  
    public String getAuthor() {  
        return author;  
    }  
  
    public void setAuthor(String author) {  
        this.author = author;  
    }  
  
    public String getPub() {  
        return pub;  
    }  
  
    public void setPub(String pub) {  
        this.pub = pub;  
    }  
  
    public int getPrice() {
```



```

        return price;
    }

    public void setPrice(int price) {
        this.price = price;
    }
}

```

Service class

```

package org.techhub.service;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.techhub.model.Book;
import org.techhub.repo.BookRepo;

@Service("bookService")
public class BookService {
    @Autowired
    BookRepo bookRepo;
    public boolean isAddBook(Book book) {
        //boolean b= bookRepo.isAddBook(book);
        //return b;
        return bookRepo.isAddBook(book);
    }
    public List<Book> getAllBook(){
        return bookRepo.getAllBook();
    }
    public Book searchBook(String bookName,String authorName) {
        return bookRepo.searchBook(bookName, authorName);
    }
    public boolean isDelete(int bookId) {
        return bookRepo.isDelete(bookId);
    }
}

```

BookRepository

```

package org.techhub.repo;
import java.sql.PreparedStatement;

```

```

import java.sql.ResultSet;
import java.sql.SQLException;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.jdbc.core.PreparedStatementSetter;
import org.springframework.jdbc.core.RowMapper;
import org.springframework.stereotype.Repository;
import org.techhub.model.Book;
import java.util.*;

@Repository("bookRepo") //bookRepo work as reference or bean id
public class BookRepo {
    @Autowired
    JdbcTemplate template;

    List<Book> list;
    public boolean isAddBook(final Book book) {
        //write here insert logic using template.
        /*PreparedStatementSetter pstmt= new PreparedStatementSetter() {

            public void setValues(PreparedStatement ps) throws SQLException {

                ps.setString(1,book.getName());
                ps.setString(2, book.getAuthor());
                ps.setString(3, book.getPub());
                ps.setInt(4,book.getPrice());
            }
        };*/
        //below code is lamda version of above code.
        PreparedStatementSetter pstmt= (PreparedStatement ps)->{

            ps.setString(1,book.getName());
            ps.setString(2, book.getAuthor());
            ps.setString(3, book.getPub());
            ps.setInt(4,book.getPrice());
        };

        int value=template.update("insert into book values('0',?,?,?,?)", pstmt);
        return value>0?true:false;
    }

    public List<Book> getAllBook()
    { RowMapper<Book> r = new RowMapper<Book>() {

```

```

        @Override
        public Book mapRow(ResultSet rs, int rowNum) throws SQLException {
            Book b = new Book();
            b.setId(rs.getInt(1));
            b.setName(rs.getString(2));
            b.setAuthor(rs.getString(3));
            b.setPub(rs.getString(4));
            b.setPrice(rs.getInt(5));
            return b;
        }
    };
    list=template.query("select *from book", r);
    return list;
}

public Book searchBook(String bookName, String authorName) {

    PreparedStatementSetter st = new PreparedStatementSetter() {

        @Override
        public void setValues(PreparedStatement ps) throws SQLException {
            // TODO Auto-generated method stub
            ps.setString(1, bookName);
            ps.setString(2, authorName);
        }

    };

    RowMapper r = new RowMapper<Book>() {

        @Override
        public Book mapRow(ResultSet rs, int rowNum) throws SQLException {
            // TODO Auto-generated method stub
            Book b = new Book();
            b.setName(rs.getString(2));
            b.setAuthor(rs.getString(3));
            b.setPub(rs.getString(4));
            b.setPrice(rs.getInt(5));
            return b;
        }

    };
    List<Book> list = template.query("select *from book where name=? and author=?", st,
r);

```

```

        return list.size()>0?list.get(0):null;
    }
    public boolean isDelete(int bookid) {
        //int value= template.update("delete from book where bookid="+bookid);
        //return value>0?true:false;

        return template.update("delete from book where bookid="+bookid)>0?true:false;
    }
}

```

ClientApplicaton

```

package org.techhub.client;

import org.springframework.context.annotation.AnnotationConfigApplicationContext;
import org.techhub.config.ConfigApp;
import org.techhub.model.Book;
import org.techhub.service.BookService;
import java.util.*;

public class ClientApplication {
    public static void main(String[] args) {

        AnnotationConfigApplicationContext context= new
            AnnotationConfigApplicationContext(ConfigApp.class);
        BookService bookService=(BookService)context.getBean("bookService");
        do {
            Scanner xyz = new Scanner(System.in);
            System.out.println("1:Add New Book");
            System.out.println("2:View All Book");
            System.out.println("3:Search Book by Name or Author");
            System.out.println("4:delete Book");
            System.out.println("Enter your choice");
            int choice=xyz.nextInt();
            switch(choice) {
                case 1:
                    xyz.nextLine();
                    System.out.println("Enter name author publication and price of book");
                    String name=xyz.nextLine();
                    String author=xyz.nextLine();
                    String pub=xyz.nextLine();
                    int price=xyz.nextInt();
                    Book b = new Book(name,author,pub,price);
                    boolean b1= bookService.isAddBook(b);

```

```

        if(b1) {
            System.out.println("New Book Added");
        }
        else {
            System.out.println("Some problem is there.....");
        }
        break;
case 2:
    List<Book> list=bookService.getAllBook();

    list.forEach((book)->System.out.println(book.getName()+"\t"+book.g
etAuthor()+"\t"+book.getPrice()));
        break;
case 3:
    xyz.nextLine();
    System.out.println("Enter book name");
    String bookName=xyz.nextLine();
    System.out.println("Enter author name");
    String authName=xyz.nextLine();
    Book book=bookService.searchBook(bookName, authName);
    if(book!=null) {

System.out.println(book.getName()+"\t"+book.getAuthor()+"\t"+book.getPub()+"\t"+book.getPrice());
        }
        else {
            System.out.println("Book Not Found");
        }
        break;
case 4:
    int bookId=xyz.nextInt();
    b1=bookService.isDelete(bookId);
    if(b1) {
        System.out.println("Book Deleted Success.....");
    }
    else {
        System.out.println("Some problem is there.....");
    }
    break;
case 5:
    break;
case 6:
    System.exit(0);

```

```
        break;
    default:
        System.out.println("Wrong choice");
    }
}while(true);
}
```